

Задача 8. (13 т.) Да се реализира подходящо представяне на двоично дърво за търсене. За избраното представяне да се реализират операциите *добавяне на елемент* и *търсене на елемент по стойност*. Да се състави функция, която по дадено такова дърво извлича в нарастващ ред всички елементи, които са на определена дълбочина.

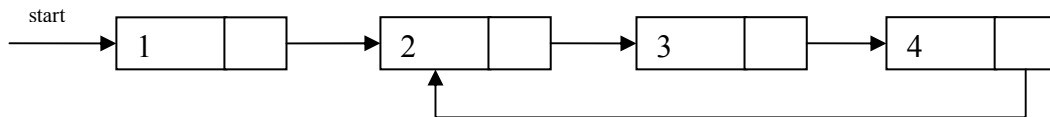
13.09.2008г. ДИ ОКС „Бакалавър” по Компютърни науки, СУ-ФМИ фак. № стр. 3/10

Задача 3. (15 т.) Да се реализира подходящо представяне на граф. За така представения граф да се реализира функция, която намира всички върхове, до които има път от подаден връх с определена дължина *в брой ребра*.

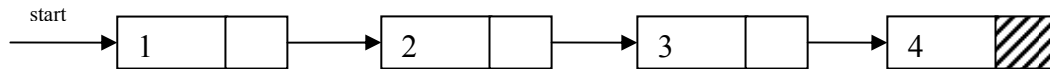
Задача 2. (15т.) Да се избере и опише представяне на граф, ребрата на който са надписани със символи. Напишете програма, която прочита от текстов файл информация за графа и я съхранява чрез описаното представяне. На първия ред на файла са записани две числа разделени с празен символ – броя на върховете и броя на ребрата на графа. Следва по един ред за всяко ребро, съставен от две числа и символ, разделени също с празен символ, номерата на върховете съответно в началото и края на реброто и надписът на реброто. За този граф, при подаден начален връх да се намерят всички други върхове, достижими от него, така че пътят до тях не съдържа ребра с повтарящи се надписи.

Задача 4. (15 точки) Да се дефинира тип данни, описващ възел в линеен едносвързан списък от цели числа. Да се напише булева функция, която проверява дали в даден едносвързан списък съществува цикъл. Цикъл наричаме такава редица от указатели към съседни възли, в която поне един указател се среща повече от веднъж.

Пример: В списъка на фиг. 1 съществува цикъл, а в списъка на фигура 2 – не съществува.



Фиг. 1



Фиг. 2

Задача 5. (15 точки) Да се дефинира тип данни, описващ възел на двоично дърво от цели числа. Да се напише булева функция, която проверява дали дадено двоично дърво е балансирано.

Забележка. Едно двоично дърво е балансирано, ако за всеки негов връх v , височините на лявото и дясното поддървета на v се различават най-много с 1.

Задача 4. (15 точки) Класът DLLlist, представя списък с две връзки, съдържащ числа с плаваща запетая. Дефиницията на класа е следната:

```
struct node {
    double data;
    node* next;
    node* prev;
};

class DLLlist {
private:
    node* start;           // указател към началото на списъка
    node* end;             // указател към края на списъка
    node* forwardIter;     // итератор за обхождане напред
    node* backwardIter;    // итератор за обхождане назад
public:
    // функции от голямата четворка
    DLLlist();
    DLLlist(DLLlist const&);
    DLLlist& operator=(DLLlist const&);
    ~DLLlist();
    // вмъква числото x пред клетката p
    void insertBefore(node* p, double x);
    // вмъква числото x след клетката p
    void insertAfter(node* p, double x);
    // премахва клетката p от списъка
    void insertAfter(node* p, double x);
    // установява forwardIter да сочи към началото на списъка
    // или към клетката p, ако ѝ е зададена ненулева стойност
    void startForward(node* p=NULL);
    // установява backwardIter да сочи към началото на списъка
    // или към клетката p, ако ѝ е зададена ненулева стойност
    void startBackward(node* p=NULL);
    // премества forwardIter напред и връща старата му стойност
    node* nextForward();
    // премества backwardIter напред и връща старата му стойност
    node* nextBackward();
};
```

а) (3 т.) Да се дефинират член-функциите от голямата четворка;

б) (3 т.) Да се дефинират член-функциите insertBefore и insertAfter;

в) (2 т.) Да се дефинира член-функцията `deleteElem`;

г) (3 т.) Да се дефинира функцията `bool mirror(DLList& dl1, DLList& dl2)`, която проверява дали списъкът `dl1` е огледален образ на `dl2`, т.е. дали `dl1` се състои от елементите на `dl2` в обратен ред;

д) (4 т.) Да се дефинира функцията `node* split(DLList& dl, double x)`, която с едновременно обхождане от началото и края нарежда елементите на списъка `dl` в две области така, че в началото на списъка да се окажат само числа, по-малки от x , а в края на списъка — числа, по-големи от x . За целта да се използва следния алгоритъм: списъкът `dl` се обхожда от двата края до момента, в който бъдат достигнати два елемента a (от началото) и b (от края), такива че $a > x > b$. При достигане на такива елементи те да бъдат разменени и обхождането да продължи със същата стратегия. Като резултат да бъде върната границата между двете области, т.е. указател към клетката, съдържаща последния по ред елемент в `dl`, който е по-малък от x .

Задача 4. (12 точки) Да се дефинира клас `BinTree`, който представя двоично дърво с наредба на синовете и ориентация на ребрата от родител към дете, надписано със символи по върховете. **Ориентираният път** между два върха на дървото се представя с низа от надписите на последователните върхове на пътя.

а) (2 т.) Да се дефинират член-данни и помощни структури за класа `BinTree`, които са необходими за избраното представяне.

б) (3 т.) Да се реализира член-функция `isSymmetric()`, която проверява дали двоичното дърво е симетрично относно наредбата на синовете (ляво-дясно), т.е. дали съвпада с огледалния си образ.

в) (3 т.) Да се реализира член-функция `appendTree(t)` която вмъква дадено двоично дърво `t` на мястото на всички листа на дървото, чиито надписи съвпадат с надписа в корена на `t`.

г) (4 т.) Да се реализира член-функция `contains(s)`, която проверява дали даден низ `s` представя път в дървото.

Задача 6. (10 т.) *Задачата да се реши на езика C++ или Java. В началото на вашето решение посочете кой език сте избрали.*

Едносвързан цикличен списък от цели числа се описва с референция (указател) към циклична верига от двойни клетки, представени по следния начин:

C++

```
struct Node {  
    Node *next;  
    int data;  
};
```

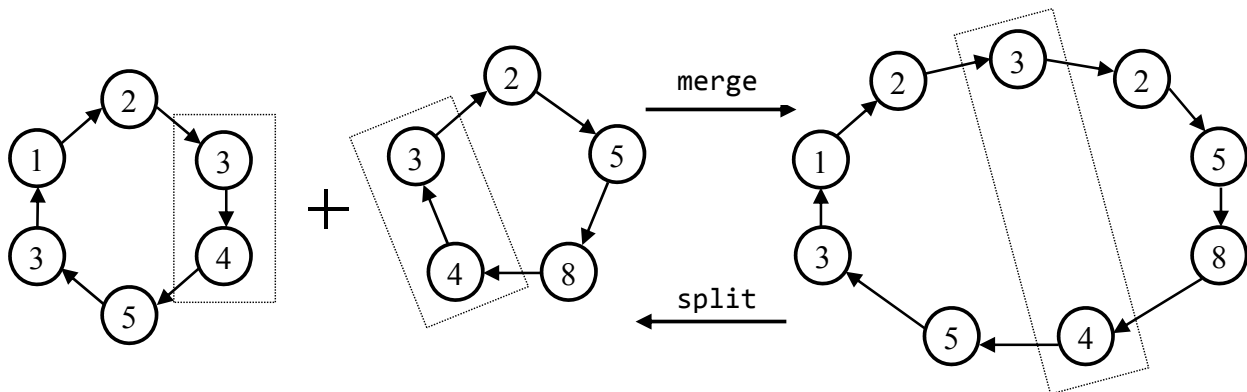
Java

```
public class Node {  
    public Node next;  
    public int data;  
}
```

А) За даден цикличен списък L казваме, че числото M предхожда числото N , ако в списъка L има кутия A , която съдържа M , а $A.next$ сочи към кутия, която съдържа N .

Да се реализира (статична) функция `precedes`, която по даден списък L и две числа M и N проверява дали M предхожда N в L и ако е така, връща референция (указател) към кутията A , която съдържа M .

Б) Ако са дадени два списъка $L1$ и $L2$, така че M предхожда N в $L1$ и N предхожда M в $L2$, тогава двата списъка могат да се слоят в един списък L , както е показано на диаграмата долу, така че в L остава само по един екземпляр на числата M и N .



Да се реализира (статична) функция `merge`, която слива два списъка $L1$ и $L2$, ако това е възможно. В случай, че сливането може да стане по няколко различни начина, да се избере такъв, за който сумата на числата M и N е максимална.

В) Да се реализира (статична) функция `split`, която по даден списък L и числа M и N , ако е възможно, разделя списъка L на два списъка $L1$ и $L2$, така че M предхожда N в $L1$ и N предхожда M в $L2$, както е показано на диаграмата горе. Ако разделянето може да стане по повече от един начин, да се избере такъв, при който разликата между дължините на получените списъци $L1$ и $L2$ е минимална.

Задача 6. (10 т.) Задачата да се реши на езика C++ или Java. В началото на вашето решение посочете кой език сте избрали.

Двусвързан списък от цели числа се описва с двойка референции (указатели) съответно към началото и края на двусвързана верига от тройни клетки, представени по следния начин:

C++

```
struct Node {  
    Node *next, *prev;  
    int data;  
};
```

Java

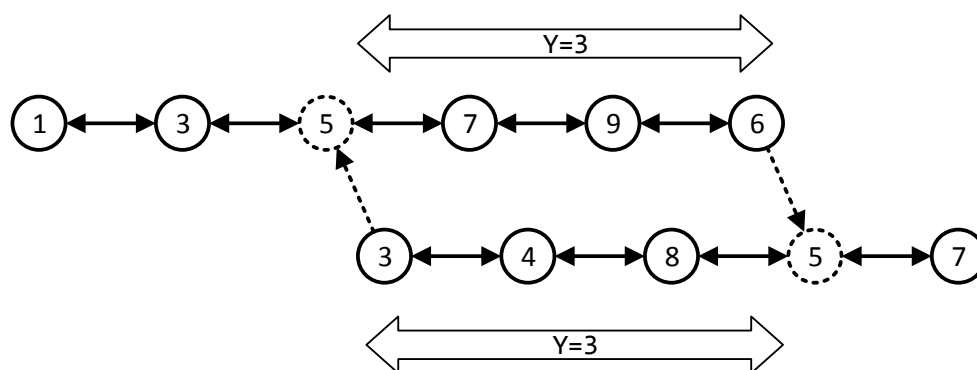
```
public class Node {  
    public Node next, prev;  
    public int data;  
}
```

Казваме, че двусвързаният списък L1 може да бъде „снаден“ с двусвързания списък L2 в числото M, ако има кутия A в L1 и кутия B в L2, такива че:

- A.data и B.data са равни на M;
- A е на разстояние Y от края на L1;
- B е на разстояние Y от началото на L2.

Разстояние между две кутии е броят на връзките, през които се преминава от едната до другата кутия. Снаждането на списъците се осъществява както е показано на диаграмата по-долу, като получената структура наричаме „снаден списък“.

Пример: Снаден списък, получен от снаждането на два списъка в числото 5 при Y = 3



А) Да се реализира функция join, която „сnaжда“ два двусвързани списъка L1 и L2, ако това е възможно. В случай, че снаждането може да се получи по няколко различни начина, да се избере този, при който разстоянието Y е минимално.

Б) Да се реализира булева функция isJoined, която по двойка референции (указатели) към начало и края на двусвързана верига проверява дали веригата е снаден списък. **Забележка:** да се счита, че подадената верига е двусвързан или снаден списък, т.е. не е нужно функцията да може да обработва друг вид вериги.

В) Да се реализира функция sum, която по даден снаден списък намира сумата на всичките му елементи.

Забележка: Ако решавате задачата на езика Java, за всяко подусловие е достатъчно да реализирате статична функция, която получава вход съгласно спецификацията и извършва нужните действия.

Задача 2 (10 точки). Следната задача да се реши на един от езиците за програмиране C++ или Java. Да се обозначи явно на кой от двата езика е решавана задачата. При решението на задачата да не се използват библиотеки за работа със структури от данни.

а) Да се дефинира подходяща *индуктивна (рекурсивна)* структура от данни, позволяваща представянето в паметта на програмата на възел на дърво от цели числа (`int`), за което всеки връх може да има произволен брой наследници (0, 1 или повече).

б) Да се дефинира *рекурсивна* функция (или статичен метод)

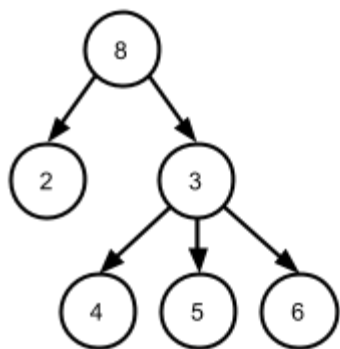
```
[булев тип] member ([подходящ тип] root, int x)
```

чиято стойност е истина точно тогава, когато в дървото с корен, представен от параметъра `root`, съществува възел със стойност `x`.

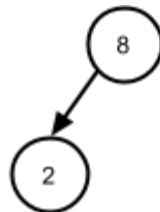
в) Да се дефинира рекурсивна функция (или статичен метод)

```
void filterOdd ([подходящ тип] root)
```

Функцията да премахва (чрез мутация) всяко поддърво `t'` на дървото с корен, представен от параметъра `root`, за което е изпълнено, че коренът на `t'` е със стойност нечетно число. На следната фигура е показано примерно дърво преди и след изпълнението на операцията `filterOdd`.



примерно дърво *t*



дървото *t* след
приложение на `filterOdd`

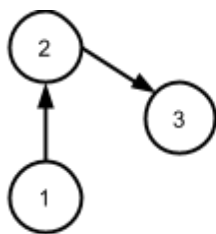
Задача 2 (10 точки). Следната задача да се реши на един от езиците за програмиране C++ или Java. Да се обозначи явно на кой от двата езика е решавана задачата. При решението на задачата да не се използват библиотеки за работа със структури от данни.

а) Да се дефинира подходяща структура от данни, позволяваща представянето в паметта на програмата на ориентиран граф от типа $G = \langle V, E \rangle$, където V е множеството на целите положителни числа, не по-големи от 1000 (представено чрез типа данни `int`), а $E = V \times V$.

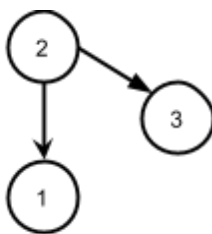
б) За така дефинираната структура от данни да се дефинира функция (или статичен метод)

`[булев тип] isConnected ([подходящ тип] g)`

чиято стойност е истина точно за тези графи g , които са свързани. За един граф $G = \langle V, E \rangle$ казваме, че е свързан, ако за всяка двойка $u \in V$, $v \in V$ има път от u до v или от v до u . На примера са показани два графа, първият от които е свързан, а вторият – не.



свързан
ориентиран граф



несвързан
ориентиран граф

Забележка: При избор на Java за език за програмиране е достатъчно да се дефинира статичен метод, който решава задачата.

Задача 2. Задачата да се реши с използване на език за процедурно или обектно-ориентирано програмиране (C, C++ или Java).

Нека в текстов файл се съдържа единствен ред текст, в който е описано представяне на дърво по следната схема:

$\langle \text{дърво} \rangle ::= (\langle \text{корен} \rangle (\langle \text{празен низ} \rangle | \langle \text{списък от наследници} \rangle)),$

където

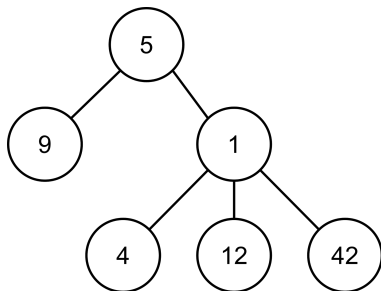
$\langle \text{корен} \rangle ::= \langle \text{цяло число} \rangle$

$\langle \text{списък от наследници} \rangle ::= \langle \text{дърво} \rangle | \langle \text{дърво} \rangle, \langle \text{списък от наследници} \rangle$

В текста се допуска наличието на произволен брой интервали между отделните елементи.

Да се напише функция, която получава като параметър низ, задаващ име на текстов файл, описващ дърво съгласно горната схема. Функцията да прочита съдържанието на файла, да построява в паметта дърво, съответстващо на текстовото описание, и да извежда на конзолата максималната сума от стойностите по път от корена на дървото до някое от листата му. Конкретното представяне на дървото в паметта е по Ваш избор.

Пример: Дървото, показано на долната диаграма, се представя по следния начин:



$(5 ((9 ()), (1 ((4 ()), (12 ()), (42 ())))))$

За този пример функцията следва да изведе на екрана числото 48
 $= 5 + 1 + 42.$

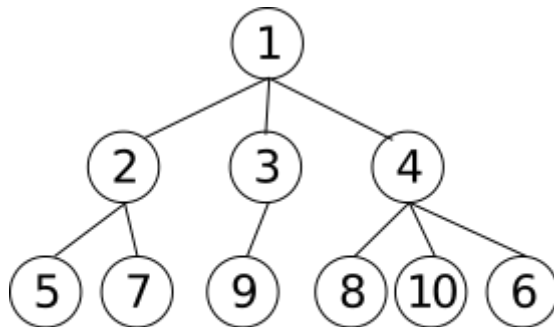
Задача 2. Задачата да се реши с използване на език за процедурно или обектно-ориентирано програмиране (C, C++ или Java).

Да се напише функция, която получава като параметри цяло число K и едномерен масив A с елементи различни цели числа. Функцията трябва да построи в паметта дърво T , съдържащо данните в масива A , като дървото T трябва да удовлетворява следните условия:

1. Всеки елемент на A се среща като възел в T точно веднъж.
2. Всеки възел в T има най-много K преки наследници (деца).
3. T е с възможно най-малка дълбочина.
4. Ако $i < j$, то $A[i]$ да не се намира по-дълбоко от $A[j]$ в T (т.е. да е на същата или по-малка дълбочина).

Като резултат функцията да връща построеното дърво и да извежда на стандартния изход неговата дълбочина. Конкретното представяне на дървото в паметта е по Ваш избор.

Пример: За $A = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ и $K = 3$, минималната дълбочина е **3**, а едно дърво T , удовлетворяващо горните условия, е:



Задача 2. (25 min)

Напишете функция, която получава като аргумент свързан списък с елементи цели числа и го сортира. Списъкът е представен и подаден на функцията по избран от вас начин.

Напишете кратка програма, която създава списък, добавя в него няколко елемента, използва функцията за сортиране и извежда на екрана резултата.

Използвайте езиците JAVA, C или C++, но без библиотечни структури данни и алгоритми.

Задача 2. (30 min)

Напишете функция, която получава като аргумент свързан списък с елементи свързани списъци от цели числа. Някои от тези списъци ще са сортирани, други не. Вашата задача е да върнете като резултат нов сортиран списък от цели числа, получен чрез сливането на всички сортирани списъци.

Напишете кратка програма, която демонстрира използването на горната функция.

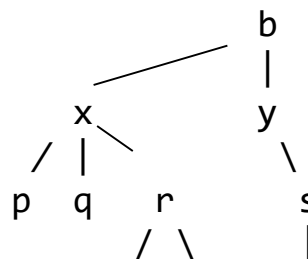
Използвайте езиците JAVA, C или C++, но без библиотечни структури данни и алгоритми.

Задача 2. Задачата да се реши на един от езиките C/C++ или Java. В началото на решението си посочете кой език сте избрали.

Троично дърво от латински букви ще наричаме следната структура:

- Празното дърво е троично дърво от латински букви;
- Ако T_1 , T_2 и T_3 са троични дървета от латински букви, а x е латинска буква, то наредената четворка $\langle x, T_1, T_2, T_3 \rangle$ също е троично дърво от латински букви.

а) Да се напише функция **readLast**, която по дадено троично дърво от латински букви намира думата, която може да се прочете отляво надясно в последното му ниво. Ниво на троично дърво от латински букви наричаме списък от възли в дървото, които са равноотдалечени от корена.



Пример: за дървото вдясно функцията **readLast** трябва да връща думата “cat”.

Представянето на дървото е по Ваш избор. Опишете избраното от Вас представяне.

б) Да се напише функция **serialize**, която по дадени: (1) троично дърво от латински букви и (2) низ, описващ път до файл, записва дървото в текстов файл. Текстовото представяне на троично дърво да бъде следното:

- Текстовото представяне на празното дърво е звездичка (“*”);
- Текстовото представяне на троичното дърво $\langle x, T_1, T_2, T_3 \rangle$ е “(x T1 T2 T3)”, където T_1 , T_2 и T_3 са текстовите представяния съответно на T_1 , T_2 и T_3 .

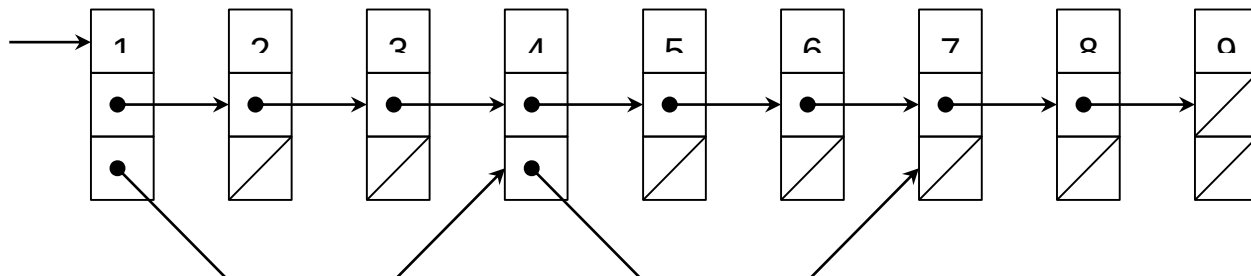
Пример: Текстовото представяне на дървото горе е:

(b (x (p * * *) (q * * *) (r (c * * *) * (a * * *))) (y * * (s * (t * * * *) *))) *

За реализацията на функциите **readLast** и **serialize** е позволено използването на стандартните за съответния език библиотечни функции.

Задача 2. Задачата да се реши на един от езиките C, C++ или Java. В началото на решението си посочете кой език сте избрали.

Нека разгледаме следната структура от данни за бързо търсене, съхраняваща сортирана редица от цели числа:



Нека имаме списък с n елемента и нека с k означим горната цяла част на числото $\sqrt{n}$. Всеки възел на списъка съдържа цяло число и два указателя. Първият указател винаги сочи следващия елемент (ако има такъв). За възлите, намиращи се на индекс, кратен на k , вторият указател сочи възела, намиращ се на k позиции напред (ако има такъв). В горния пример $n = 9$, $k = 3$.

а) Да се напише функция **readList**, която по подаден път до текстов файл прочита от файла сортирана редица от цели числа и конструира списък от описания по-горе тип, съдържащ прочетените числа. Числата са записани във файла на един ред и са разделени с интервали.

б) Да се напише булева функция **member**, която по даден списък от описания тип и дадено цяло число проверява дали това число се съдържа в списъка. Функцията да реализира ефективен алгоритъм за търсене, който се възползва от особеностите на структурата, за да минимизира броя на обходените елементи.

Пример: ако в списъка от диаграмата по-горе търсим числото 6, при ефективния алгоритъм за търсене ще бъдат последователно обходени следните елементи: 1, 4, 7, 5, 6 и функцията **member** ще върне резултат "истина".

За реализацията на гореописаната структура от данни и за функциите **readList** и **member** не е позволено използване на библиотечни структури от данни и алгоритми, но е позволено използването на стандартните функции за работа с файлове.

Задача 2. Задачата да се реши на един от езиките C, C++ или Java. В началото на решението си посочете кой език сте избрали.

Разглеждаме кореново дърво, във възлите на което има записани двойки от символ (`char`) и цяло число (`int`). Всеки възел на дървото може да има произволен, краен брой наследници. За удобство разглеждаме функциите *sym* и *val*, дефинирани над множеството от възлите на дървото, така че за всеки възел *v* на дървото, в който е записана двойката $\langle a, b \rangle$, $\text{sym}(v) = a$ и $\text{val}(v) = b$.

Клон в дървото *T* ще наричаме всеки път $\pi = (v_0, v_1, \dots, v_n)$, за който $v_0, \dots, v_n$ са върхове на *T*, v_n е листо на *T*, а v_i е родител на v_{i+1} за всяко $i < n$. За всеки клон $\pi = (v_0, v_1, \dots, v_n)$ на дървото *T* дефинираме съответни “дума” и “стойност” по следния начин:

$\text{word}(\pi) = \text{sym}(v_0)\text{sym}(v_1)\dots\text{sym}(v_n)$,

$$\text{value}(\pi) = \sum_{i=0}^n \text{val}(v_i),$$

т.е. $\text{word}(\pi)$ е думата, която се получава от последователното прочитане на символите, записани във възлите на пътя, а $\text{value}(\pi)$ е сумата на числата, записани в тях.

- а) Да се избере, дефинира и опише подходящо представяне на дърво от описания тип.
б) За така дефинираното представяне да се реализира функцията:

`int sumVal ([подходящ тип] T, [подходящ тип] u, [подходящ тип] v),`

която по дадено дърво *T* и два негови върха *u* и *v* намира и връща сумата от всички стойности $\text{value}(\pi_u) + \text{value}(\pi_v)$, за които π_u и π_v са клони с начала *u* и *v* (съответно) със свойството $\text{word}(\pi_u) = \text{word}(\pi_v)$. Ако такива клони няма, сумата се счита за 0.

Забележки:

1. В зависимост от избраното представяне, параметърът *T* може да бъде пропуснат.
2. Не е нужно дефиницията на представянето на дървото да бъде пълна, нужно е само да е достатъчна за реализацията на функцията **sumVal**.
3. Позволено е използването на функциите и класовете от стандартната библиотека на избирания от Вас език.

Задача 2. Задачата да се реши на един от езиците C, C++ или Java. В началото на решението си посочете кой език сте избрали.

Дърво с етикети ще наричаме кореново дърво $T = (V, E, r)$ с множество от върхове V , множество от ребра E и корен r , за което са дефинирани две допълнителни функции:

$$value : E \rightarrow \{n \in \mathbb{N} \mid 0 \leq n < 2^{32}\} \quad \text{и} \quad label : E \rightarrow \{a, b, \dots, z\}.$$

Клон в T ще наричаме път $\pi = (v_0, v_1, \dots, v_n)$, за който v_n е листо на T и v_i е родител на v_{i+1} за всяко $i < n$.

За всеки клон $\pi = (v_0, v_1, \dots, v_n)$ дефинираме:

$$val(\pi) = \sum_{i=0}^{n-1} value(\langle v_i, v_{i+1} \rangle) \quad \text{и} \quad word(\pi) = label(\langle v_0, v_1 \rangle) label(\langle v_1, v_2 \rangle) \dots label(\langle v_{n-1}, v_n \rangle).$$

А) Да се избере, дефинира и опише подходящо представяне на дърво от описания вид.

Б) За така дефинираното представяне да се реализира функцията:

CommonBranches(<labeled tree> T, <vertex> u, <vertex> v, <integer> k),

която по дадено дърво с етикети $T = (V, E, r)$, два негови върха u и v и естествено число k извежда на стандартния изход всички думи w , за които има клони π_u и π_v с начало u и съответно v , за които $word(\pi_u) = word(\pi_v) = w$ и $val(\pi_u) + val(\pi_v) = k$.

Забележки:

- Отделните думи да са разделени със символа за нов ред. Една дума може да се извежда повече от веднъж.
- Функционалности на структурата дърво с етикети, които нямат отношение към задачата и не се използват във функцията CommonBranches, няма да бъдат оценявани.

Задача 2. Цикличен едносвързан списък наричаме линеен едносвързан списък, в който указателят за следващ елемент на последния елемент сочи към първия елемент, вместо да е нулев. Да се реализира функция `void unite(Node* list)`, която получава като параметър указател към елемент на цикличен едносвързан списък от символни низове, описан от следната структура:

```
struct Node {  
    std::string text;  
    Node* next;  
};
```

Функцията да обединява всички двойки последователни елементи на списъка, за които последният символ на единия елемент съвпада с първия символ на непосредствено следващия го елемент, в общ елемент, чийто низ е съставен от слепването на низовете на двата елемента, разделени с тире.

Да се демонстрира работата на функцията в кратка програма, която прочита низове от стандартния вход (по един на ред), добавя ги в съответния цикличен списък, подава го на функцията `unite` и извежда последователно елементите на променения списък на стандартния изход, започвайки от лексикографски най-малкия низ.

Пример:

Вход:

```
street  
taxi  
ink  
dog  
smile  
eat  
tall  
pass
```

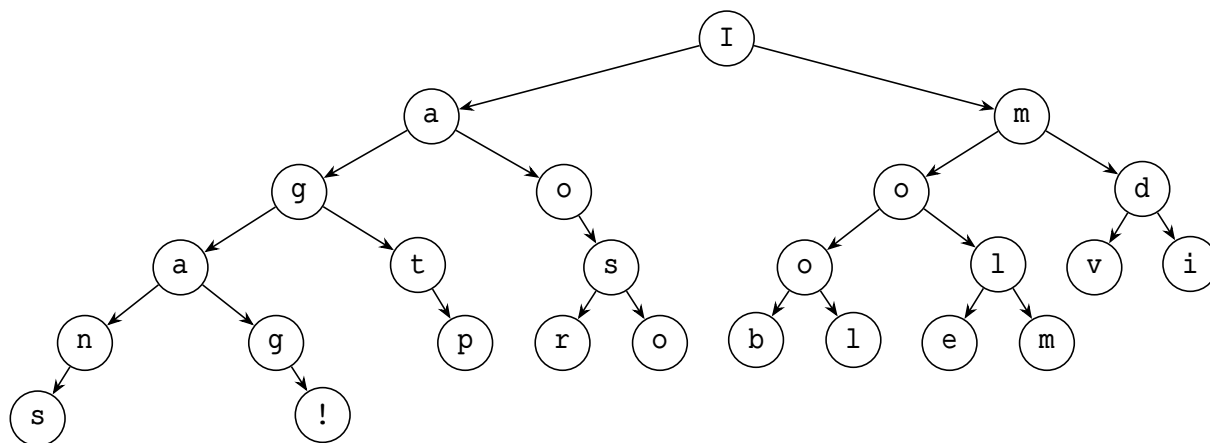
Изход:

```
dog  
smile-eat-tall  
pass-street-taxi-ink
```

Задача 2. Разглеждаме **двоично дърво** с данни от тип символ и елементи, описвани от следната структура:

```
struct Node {
    char text;
    Node *left, *right;
};
```

Стойностите са малки и главни латински букви, цифри и препинателни знаци. Няма празни символи (интервал, нов ред, табулация) и символи от разширената ASCII таблица (с код по-голям от 127). Пример за такова дърво е:



Да се реализира функция `printText`, която получава като аргумент указател към корен на такова дърво и извежда на стандартния изход текста, записан във върховете, спазвайки следните правила:

- съдържанието на всяко ниво (елементи с еднаква дълбочина) се извежда на отделен ред;
- ако на дадено място в нивото липсва възел, да се изведе интервал;
- дървото се извежда до последния елемент от последното ниво.

За показаното като пример дърво трябва да се изведе следния текст:

```
|I|
|am|
|good|
|at_solvi|
|ng_p__roblem____|
|s__!|
```

Забележка: за яснота на примера в началото и края на всеки ред е добавен символът `pipe` `|`, който `printText` не трябва да извежда.

Да се демонстрира работата на тази функция в кратка програма.

Забележка: позволено е използването на класовете контейнери от стандартната библиотека `STL`.

Задача 2. Разредена матрица представлява матрица с голяма размерност, за която малък брой от елементите са ненулеви(*).

- А) Опишете с думи подходяща структура от данни, в която да съхранявате такава матрица.
- Б) Напишете функция, която приема като аргументи две разредени матрици от реални числа, представени по описания от вас начин, и връща като резултат тяхната сума, отново представена по този начин.
- В) Демонстрирайте тази функция в кратка програма.

(*)За матрица с голяма размерност приемаме, например, матрица с размерност $10^6 \times 10^6$ и брой на ненулевите елементи от порядъка на 10^4 .

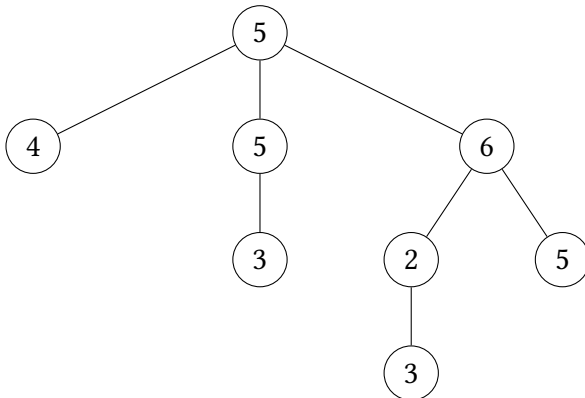
За задачката се предполага използването на езика C или C++. Разрешава се използването на стандартните за езика библиотеки, като се изисква да се посочи кой език и съответно коя негова версия е използвана.

Задача 2. Разглеждаме (кореново) дърво с върхове, чиито стойности са цели числа, като всеки връх може да има произволен брой деца. Такова дърво с $k + 1$ върха можем да представим като редица от двойки цели числа

$(level_0, value_0), \dots, (level_k, value_k),$

където за всяко $i : 0 \leq i \leq k$, $level_i$ показва нивото в дървото, на което се намира i -тият връх от представянето, а $value_i$ — неговата стойност. $(level_0, value_0)$ е коренът на дървото ($level_0 = 0$), а родителят на всеки връх $(level_i, value_i), i > 0$, е най-близкият до него предходен елемент в редицата $(level_j, value_j), 0 \leq j < i$, такъв че $level_j + 1 = level_i$.

Пример: Редицата $(0, 5), (1, 4), (1, 5), (2, 3), (1, 6), (2, 2), (3, 3), (2, 5)$ описва следното дърво:



Разбира се, не всяка такава редица е коректно представяне на дърво. Например, ако имаме два върха на ниво 0, това не е валидно представяне, тъй като коренът е единствен. Друг пример на некоректна редица е такава, в която за някой връх няма предхождащ го родител (например за върха със стойност 2 в: $(0, 1), (2, 2), (1, 3)$).

А) (1 точка) Дефинирайте тип `TreeSeq` за представяне на редици от описания вид, който да ви е удобен за решаване на задачата. Можете да ползвате всички средства от стандартната библиотека.

Б) (1 точка) Дефинирайте подходящо представяне в паметта на връх на дърво от описания тип.

В) (7 точки) Напишете функция, която получава като аргумент редица от тип `TreeSeq`. Функцията да връща като резултат указател към корена на дърво, съответстващо на редицата, ако тя е валидно представяне на дърво. В противен случай да се връща `nullptr`.

Г) (1 точка) Демонстрирайте използването на тази функция в кратка програма.

Задачата да се реализира на езика C/C++.

Задача 3.

Задачата да се реши на езика C++.

В тази задача едносвързан списък ще представяме чрез указател към първата му кутия, а кутиите му – чрез структури от вида:

```
struct node {
    int data;    // числов елемент
    node* next; // следваща кутия в списъка
                // или nullptr, ако няма
    node(int data, node* next = nullptr)
        : data(data), next(next)
    {}
};
```

1) Да се довърши кодът на функциите (в подчертаните места) така, че `sortListAscending` да сортира в нарастващ ред елементите на списъка `lst`. Ако списъкът е празен, функцията да не прави нищо.

```
node* min(node* lst)
{
    node* result = lst;
    for (; lst; lst = lst->next) {
        if (result->data _____)
            result = lst;
    }
    return result;
}
```

```
void sortListAscending(node* lst)
{
    while (_____) {
        swap(_____, min(lst)->data);
        lst = _____;
    }
}
```

2) Разгледайте кода на функцията `mystery`:

```
void mystery(node* lst)
{
    while (lst && lst->next) {
        lst->data += lst->next->data;
        lst->data /= 2;
        node* tmp = lst->next;
        lst = lst->next = lst->next->next;
        delete tmp;
    }
}
```

Под дадения долу фрагмент да се напише точно какво ще изведе на стандартния изход.

```
node* e = new node(56);
node* d = new node(20, e);
node* c = new node(10, d);
node* b = new node(4, c);
node* a = new node(2, b);
mystery(a);
while (a) {
    std::cout << "[" << a->data << "]" -> " ";
    a = a->next;
}
std::cout << "NULL";
```

3) Възлите на двоично дърво представяме чрез структури от вида:

```
struct tnode {
    int data;    // числов елемент
    tnode* left; // ляв наследник
    tnode* right; // десен наследник
    // указателите са nullptr, ако възелът
    // няма съответния наследник
};
```

Да се попълнят празните места в кода на функцията `toArray`. Тя получава корена `t` на двоично наредено дърво. Функцията да копира елементите му в масива `out` така, че да са сортирани в нарастващ ред. Това да става чрез обхождане ляво-корен-дясно (`inorder`). За целите на задачата допускаме, че в `out` има достатъчно място.

```
int* toArray(tnode* t, int* out)
{
    if (_____) {
        return _____;
    }
    else {
        int* p = toArray(_____, out);
        *p = t->data;
        return toArray(_____, _____);
    }
}
```

Задача 3. Задачата да се реши на езика C++. Там, където се изисква да се посочи сложност, тя трябва да се изрази в термините на Θ нотацията. Забележка: $f(n) = \Theta(g(n))$, ако f е асимптотично ограничена отгоре и отдолу от g , например $3n^2 + n = \Theta(n^2)$, но $5n \neq \Theta(n^2) \neq 8n^3 + 2n^2$.

В тази задача едносвързан списък ще представяме чрез указател към първия му елемент. Елементите на списъка представяме чрез структури от вида:

```
struct node {  
    unsigned data;  
    node* next; // Указател към следващ елемент в списъка или nullptr, ако няма такъв.  
    node(unsigned data) : data(data), next(nullptr) {}  
};
```

1) Да се довърши кода на функцията `copyEveryOther`, така че тя да връща нов списък, в който се копират само елементите на нечетни позиции в списъка, сочен от `lst`, запазвайки техния ред. Приемаме, че позициите са номерирани от 1.

```
node* copyEveryOther(node* lst) {  
    if (!lst) return nullptr;  
    node* result = new node(_____);  
    node* p = _____;  
    while ((lst = lst->next) && (_____) ) {  
        p->next = new node(_____);  
        p = _____;  
    }  
    return result;  
}
```

2) Каква е времевата сложност на алгоритъма от 1) за списък с дължина n ?

3) Каква е външната (auxiliary) пространствена сложност на алгоритъма от 1) за списък с дължина n ? Във “външната сложност” НЕ СЕ включва пространството, консумирано от входните данни, нито това за изходните данни.

4) Нека е дадено двоично наредено дърво. То използва стандартната организация, при която за всеки връх вляво от него има по-малки, а вдясно – по-големи елементи. В началото дървото е празно. След това последователно в него се добавят числата 5, 10, -5, 1 и 12. След това елементите на дървото се обхождат “ляво-корен-дясно” (inorder) и се извеждат на екрана. В дадените по-долу полета да се попълнят елементите в реда, в който ще бъдат изведени.

_____, _____, _____, _____, _____

5) Нека е дадено произволно двоично наредено дърво, съдържащо n елемента. Каква е времевата сложност на търсенето на елемент в него в най-лошия случай?

6) Да се посочат времевите сложности на следните операции с едносвързан списък с дължина n , който ни е подаден чрез указател към първия му елемент:

Премахване на първия му елемент: _____

Премахване на последния му елемент: _____

Проверка дали в списъка се съдържа дадено число K : _____

Добавяне на число на втора позиция в списъка: _____

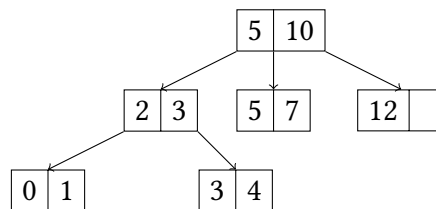
Задача 3. Троично дърво ще наричаме наредено дърво, за което всеки възел съхранява една или две стойности и има не повече от три наследника. Ако за произволен възел означим стойностите на двата елемента в него с X и Y , а трите негови потенциални поддървета с $STleft$, $STmid$ и $STright$, то за да е валидно такова дърво, трябва да са в сила следните свойства:

- стойността X трябва да е строго по-малка от стойността Y ;
- стойността на всеки елемент от поддървото $STleft$ трябва да е строго по-малка от X ;
- стойността на всеки елемент от поддървото $STmid$ трябва да е по-голяма или равна на X и строго по-малка от Y ;
- стойността на всеки елемент от поддървото $STright$ трябва да е по-голяма или равна на Y ;
- ако в даден възел няма втора стойност (Y), то тогава поддървото $STright$ задължително трябва да е празно;
- ако в поддървото $STmid$ на даден възел има поне един елемент по-голям от X , то във възела задължително трябва да има две стойности (X и Y);
- всяко от $STleft$, $STmid$ и $STright$ или е празно, или е валидно троично дърво.

Така неформално можем да напишем следното неравенство:

$$STleft < X \leq STmid < Y \leq STright$$

Пример за такова дърво е показан на фигурата:



- 1) Да се опише структура, с която може да се представи такова троично дърво с елементи цели числа и да се напише нейната дефиниция на C++.
- 2) Да се дефинира функция `insert` с аргументи троично дърво от цели числа и цяло число, която добавя това число в дървото, съблюдавайки изискванията за валидност. При добавяне на нова стойност не трябва да се променят или преместват съществуващите в дървото стойности.
- 3) Да се дефинира функция `find` с аргументи троично дърво от цели числа и цяло число, която проверява дали подаденото число се съдържа в дървото.
- 4) Да се реализира кратка програма, която демонстрира описаните функции, като за целта:
 - а) създава празно дърво;
 - б) въвежда от стандартния вход естествено число N , последвано от N цели числа, които добавя в дървото;
 - в) прочита от стандартния вход едно число и извежда на стандартния изход подходящ текст, указващ дали това число се среща или не в дървото;
 - г) подсигурява коректна работа с паметта.

Задачата трябва да се реализира на езика C++. Позволено е използването на класовете и методите от стандартната библиотека, отговаряща на стандарта C++14 или по-стар.

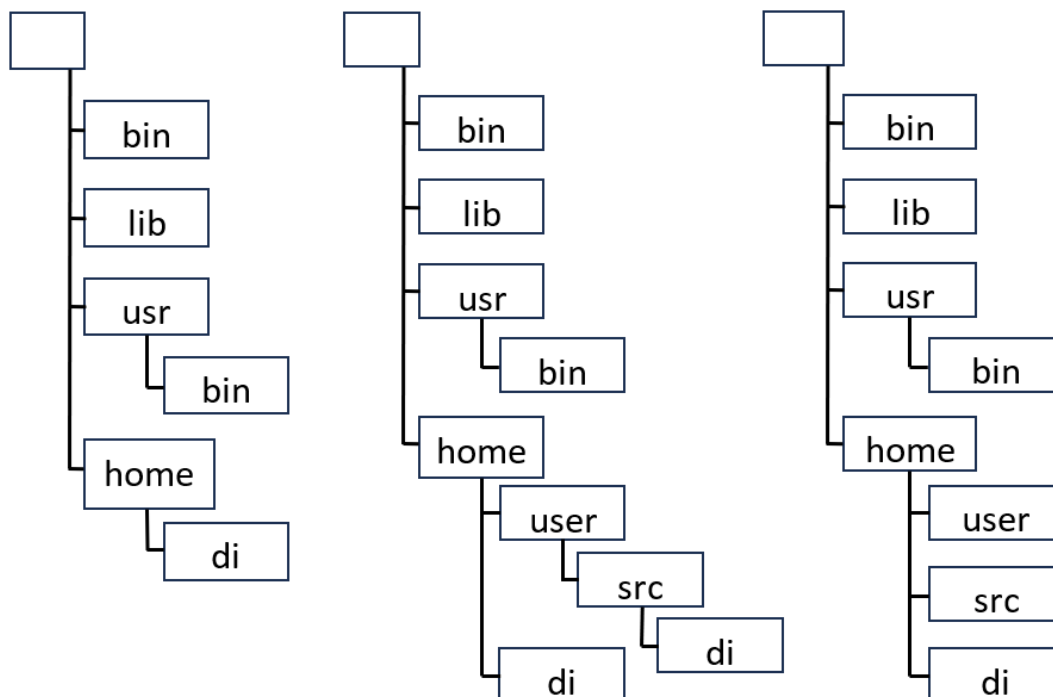
Задача 3. Разглеждаме дърво на директориите на файлова система. Всяка директория може да има произволен брой наследници. Не се допускат две директории с едно и също име и общ пряк родител (в една директория не може да има две директории с едно и също име). Всички възли имат име – символен низ с дължина не по-голяма от 12 символа, съставено от малки или главни латински букви, цифри, както и символа точка. В тази задача не се изисква да се разглеждат други характеристики на директориите.

Корен на дървото винаги е главната директория. Тя е единственият елемент с празно име.

1. Да се опише структура, с която можете да се представи такова дърво и да се напише нейната дефиниция на C++.
2. Да се дефинира функцията `insert`, която получава като аргументи дърво на файлова система и символен низ, описващ пълен път към директория, като за разделител се използва символът `'/'`. Функцията трябва да добави тази директория в дървото, като при нужда създава всички липсващи родителски директории от пълния път до нея. Например, ако е зададен пълен път `"/home/user/src"`, след приключване на работата на функцията в дървото със сигурност трябва да има елемент с име `"home"`, който е наследник на корена; елемент с име `"user"`, който е наследник на елемента `"home"`, и елемент с име `"src"`, който е наследник на елемента `"user"`.
3. Да се дефинира функцията `flatten`, която получава като аргумент дърво на файловата система и пълен път към елемент в него. Функцията трябва да преобразува всички непреки наследници на този елемент в негови преки наследници, т.е. всички елементи на поддървото на елемента трябва да станат преки наследници на корена му. Ако има повтарящи се имена, функцията трябва да остави само един пряк наследник с това име.

Задачата трябва да се реализира на езика C++. Позволено е използването на класовете и методите от стандартната библиотека, отговаряща на стандарта C++14 или по-стар.

Пример: На първата фигура е показано примерно дърво на директориите. На следващата към това дърво е извършено добавяне на директорията `/home/usr/src/di`. На последната е показан резултатът от прилагане на `flatten` към директорията `/home` на предходното дърво.



Задача 3. Разглеждаме (кореново) дърво с върхове, чиито стойности са цели числа, като всеки връх може да има произволен брой наследници, редът на които ще има значение – първия ще наричаме най-ляв, а последния – най-десен. „Ниво с номер n “ наричаме списък от върховете в дървото, които са на разстояние n от корена, подредени от ляво надясно. Казваме, че ниво в дървото „представя“ даден вектор от цели числа, ако елементите във върховете в нивото образуват точно елементите на вектора.

А) Да се дефинира тип `TreeNode` за представяне на връх от такова дърво.

Б) Да се реализира функция

```
int findLevel(const TreeNode* root, const vector<int>& numbers);
```

която по подаден указател към корен на такова дърво намира номер на ниво, представлящо вектора `numbers`, или `-1`, ако такова ниво няма. Ако има повече от едно ниво с такива елементи, не е от значение кой номер ще се върне.

Приемаме, че коренът определя ниво с номер 0.

В) Да се реализира функция

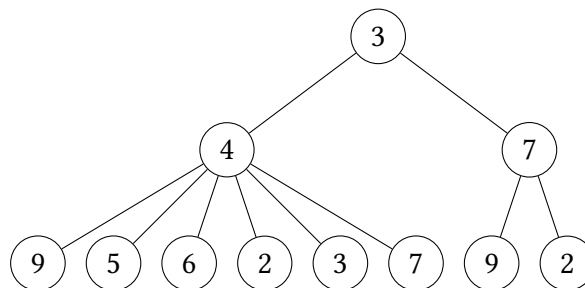
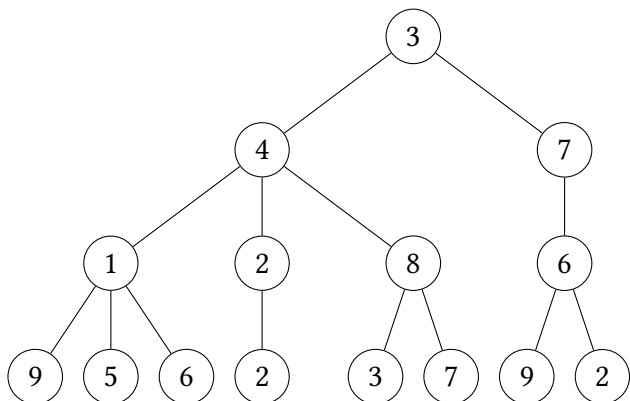
```
void removeAllLevels(TreeNode* root, const vector<int>& numbers)
```

която по подаден указател към корен на дърво премахва всички нива с положителен номер, които представят вектора `numbers`. При премахване на връх, неговите преки наследници заемат неговото място и стават преки наследници на родителя му. Приемаме, че елементите на дървото са заделени с `new`.

Задачата да се реализира на езика C/C++. Позволено е използването на всички средства от стандартната библиотека.

Пример:

За дървото вляво и вектор от числа $\{1, 2, 8, 6\}$, функцията `findLevel` трябва да върне 2, а след изпълнението на `removeAllLevels` трябва да се получи дървото вдясно.



Задача 3. *Задачата да се реши на езика C++.*

Свързан списък ще представяме с верига от обекти от следната структура, динамично заделени с new. За последния елемент на списъка указателят next има стойност nullptr. Приемаме, че за типа Type са дефинирани нужните конструктори и оператори.

```
template <typename Type>
struct node
{
    const Type data;
    node<Type>* next;
};
```

Да се реализират следните функции, получаващи като аргумент указател към първия елемент на списък и връщащи указател към началото на резултатния списък.

Не се позволява използване на алгоритми и структури от данни от стандартната библиотека.

- а) Функция, която обръща свързан списък. Тя трябва да използва **константно** количество допълнителна памет и да е с **линейна** сложност.

```
template <typename Type>
node<Type>* reverse(node<Type>* first);
```

- б) Функция, която премахва от списъка всички елементи, равни на първия при сравнение с ==, но без него. Тя трябва да използва **константно** количество допълнителна памет и да е с **линейна** сложност.

```
template <typename Type>
node<Type>* filter(node<Type>* first);
```

- в) Функция, която подрежда елементите на списъка във възходящ ред. Тя трябва да има асимптотична времева сложност в средния случай по-добра от $O(n^2)$.

Опишете накратко с текст какъв алгоритъм сте реализирали, каква е неговата времева сложност и каква е сложността му по памет.

```
template <typename Type>
node<Type>* sort(node<Type>* first);
```